CyberGuard

Phase III: Data Management & Integration Layer

Course Project Documentation

Project	CyberGuard - Enterprise Cybersecurity Platform
Phase	III - Data Management & Integration
Technology	Node.js, Express, SQLite, NeDB, JWT
Date	April 2026

Table of Contents

1. System Overview

CyberGuard Phase III extends the existing cybersecurity monitoring platform with a comprehensive data management and integration layer. Building on the Phase I React web application (16 pages) and Phase II service-oriented integration server (workflow orchestration, message queues, caching, correlation, fault handling), Phase III introduces a structured Model-View-Controller architecture with dual-database integration, RESTful APIs, asynchronous job processing, enterprise-grade caching, security hardening, data transformation services, and full observability through logging and monitoring.

The Phase III server runs on port 3002, complementing the Phase II server on port 3001 and the React frontend on port 5173. All three layers communicate via REST APIs with JSON payloads, forming a cohesive distributed system.

1.1 Architecture Diagram

```
Frontend (React + Vite, port 5173)
|-- REST API -->
Phase III Server (Express.js, port 3002)
|-- Controllers (Auth, Incidents, ThreatIntel, System)
|-- Services (Cache, AsyncProcessor, DataTransformer)
|-- Middleware (JWT Auth, Helmet, Validation, Rate Limiting)
|-- Models
|-- SQLite (Users, Incidents, Signatures, AuditLog)
|-- NeDB (ThreatIntel, SystemEvents, ScanResults, JobQueue)
|-- Winston Logger + Performance Metrics
```

2. Technology Stack Justification

Each technology was chosen based on the project requirements for modularity, security, performance, and ease of deployment.

Technology	Role	Justification
Node.js + Express	Backend Runtime & Framework	Non-blocking I/O for async processing; Express provides lightweight, flexible routing
SQLite (sql.js)	Relational Database	ACID-compliant, zero-configuration, portable. Pure JavaScript implementation (sql.js
NeDB	NoSQL Document Store	MongoDB-compatible API for semi-structured data. Embedded, file-based persisten
JWT (jsonwebtoken)	Authentication	Stateless token-based auth. Scalable across services without session storage. Inclu
Helmet	Security Headers	Sets 15+ HTTP security headers (CSP, HSTS, X-Frame-Options, etc.) to protect ag
bcryptjs	Password Hashing	Industry-standard adaptive hashing with salt rounds. Pure JavaScript, no native dep
express-validator	Input Validation	Declarative validation chains with sanitization. Prevents injection attacks at the midd
Winston	Logging	Structured JSON logging with multiple transports (file + console). Supports log levels
xml2js	Data Transformation	Bidirectional JSON-XML conversion for interoperability with external systems.
express-rate-limit	Rate Limiting	Protects APIs from abuse and DDoS. Configurable per-route limits.

3. Database Design

3.1 Relational Database (SQLite)

The SQLite database stores structured data with well-defined relationships. The schema uses foreign keys, CHECK constraints, indexes, and a normalized design following third normal form (3NF).

Users Table

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
username	TEXT	UNIQUE, NOT NULL
email	TEXT	UNIQUE, NOT NULL
password_hash	TEXT	NOT NULL (bcrypt)
role	TEXT	CHECK(admin|analyst|viewer)
is_active	INTEGER	DEFAULT 1
created_at / updated_at	TEXT	DEFAULT datetime('now')

Incidents Table

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
incident_id	TEXT	UNIQUE, NOT NULL
threat_type	TEXT	NOT NULL
severity	TEXT	CHECK(low|medium|high|critical)
source_ip	TEXT	NOT NULL
target_asset	TEXT	NOT NULL
status	TEXT	CHECK(open|investigating|...|closed)
assigned_to	INTEGER	FK -> users(id)
description	TEXT	Optional

Relationships: Users 1:N Incidents (assigned_to), Incidents M:N Threat Signatures (via incident_signatures junction table with confidence score). Audit Log references users via foreign key.

3.2 Non-Relational Database (NeDB)

NeDB stores semi-structured and variable-schema data as documents. Four collections are used:

Collection	Purpose	Key Fields
threatIntel	IOC indicators with varying types	ioc_type, ioc_value, severity, tags[], source

Collection	Purpose	Key Fields
systemEvents	Time-series system events/metrics	event_type, severity, timestamp, details{}
scanResults	Nested network scan results	scan_id, target_ip, portDetails[], vulnerabilities{}
jobQueue	Async job persistence	job_id, type, payload{}, status, attempts, priority

4. API Specifications

All endpoints follow RESTful conventions with JSON request/response bodies, standard HTTP methods, and appropriate status codes (200, 201, 400, 401, 403, 404, 429, 500).

4.1 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/v3/auth/register	None	Register new user
POST	/api/v3/auth/login	None	Login, returns JWT token

4.2 Incident CRUD Endpoints (SQLite)

Method	Endpoint	Auth	Description
GET	/api/v3/incidents	Optional	List with filters (?status, severity, limit, offset)
GET	/api/v3/incidents/:id	Optional	Get single incident by ID
POST	/api/v3/incidents	JWT	Create new incident
PUT	/api/v3/incidents/:id	JWT	Update incident fields
DELETE	/api/v3/incidents/:id	Admin	Delete incident (admin role only)
GET	/api/v3/incidents/statistics	Optional	Aggregated stats by severity/status
GET	/api/v3/incidents/:id/xml	Optional	Get incident as XML
GET	/api/v3/incidents/:id/stix	Optional	Get in STIX 2.1 format

4.3 Threat Intelligence Endpoints (NeDB)

Method	Endpoint	Auth	Description
GET	/api/v3/threat-intel	Optional	List IOCs with filters
POST	/api/v3/threat-intel	JWT	Create new IOC
PUT	/api/v3/threat-intel/:id	JWT	Update IOC
DELETE	/api/v3/threat-intel/:id	JWT	Delete IOC
GET	/api/v3/threat-intel/search?q=...	Optional	Full-text search

4.4 Response Format

All responses follow a standardized wrapper: { success: boolean, data: {...}, meta: { timestamp, version, cache } }. Errors return: { success: false, error: { message, code, details } }.

5. Data Flow Diagrams

5.1 Incident Creation Flow

```
Client POST /incidents
-> Rate Limiter (200 req/15min)
-> JWT Authentication (verify token)
-> Input Validation (express-validator)
-> Input Sanitization (XSS/SQL injection strip)
-> Controller: generate incident_id
-> Model: INSERT INTO incidents (SQLite)
-> Cache: invalidate 'incidents' tag
-> AsyncProcessor: enqueue threat_analysis job
-> AuditLog: record CREATE action
-> Response: 201 Created + incident data
```

5.2 Cached Read Flow

```
Client GET /incidents
-> Check cache (key = 'incidents:list:{query}')
-> IF cache HIT: return cached data (< 1ms)
-> IF cache MISS:
-> SQLite query with JOIN (users table)
-> Store in cache (TTL=30s, tag='incidents')
-> Response: 200 OK + data + { cache: 'miss' }
```

5.3 Async Job Processing Flow

```
Producer: POST /async/produce
-> Create job document in NeDB (status=pending)
-> Log system event

Consumer (background loop, every 5s):
-> Dequeue pending jobs (sorted by priority)
-> Execute registered handler
-> IF success: mark completed in NeDB
-> IF failure: increment attempts
-> IF max_attempts reached: move to dead_letter
-> ELSE: reset to pending (retry)
```

6. Caching and Performance Analysis

6.1 Caching Strategy

The system implements an in-memory cache service with two expiration strategies. API response caching uses absolute TTL (30 seconds) for list endpoints, ensuring fresh data within a predictable window. Individual entity caching uses sliding TTL (60 seconds), which resets on each access to keep frequently-accessed data warm.

6.2 Cache Invalidation Policy

Method	Description	Use Case
Explicit Key	invalidate(key) removes specific entry	After single entity update
Tag-Based	invalidateByTag(tag) removes all tagged entries	After any incident CRUD (tag='incidents')
TTL Auto-Eviction	Background cleanup every 30s removes expired entries	Automatic stale data removal
Manual Clear	DELETE /cache/clear wipes entire cache	Admin maintenance

6.3 Performance Comparison

The /api/v3/cache/performance endpoint runs a benchmark of 200 iterations comparing uncached (2ms simulated computation) vs. cached lookups. Typical results show cache reads at 0.002ms vs. uncached at 2.1ms, yielding approximately 99% improvement and 1000x speedup. The cache metadata is included in every API response so clients can observe hit/miss behavior.

7. Security Implementation

7.1 Authentication & Authorization

JWT (JSON Web Token) authentication is used with 24-hour token expiry. Passwords are hashed using bcrypt with 12 salt rounds. Three roles (admin, analyst, viewer) provide granular access control. Sensitive operations like DELETE require admin role.

7.2 Input Validation & Sanitization

All inputs are validated using express-validator middleware with declarative rules (required fields, format checks, length limits). A deep sanitization middleware strips HTML angle brackets, JavaScript event handlers, and SQL injection characters from all request bodies and query parameters before they reach controllers.

7.3 Protection Measures

Vulnerability	Protection	Implementation
SQL Injection	Parameterized queries + input sanitization	sql.js prepared statements with bind params
XSS	Input stripping + Helmet CSP headers	Remove angle brackets, JS handlers from input
Brute Force	Rate limiting on auth endpoints	20 requests per 15 minutes on /auth/*
API Abuse	Global rate limiting	200 requests per 15 minutes on /api/v3/*
CSRF/Clickjacking	Helmet security headers	X-Frame-Options, X-Content-Type-Options
Data Exposure	Role-based access + audit logging	Admin-only routes, all actions logged

8. Asynchronous Processing Design

The async processing system implements a producer-consumer pattern with persistent job storage in NeDB. This ensures that messages survive server restarts and can be processed after consumer downtime.

8.1 Components

Component	Description
Producer	Enqueues jobs via POST /async/produce or automatically during incident creation
Consumer	Background loop (every 5s) dequeues and processes pending jobs
Job Handlers	Registered handlers for: threat_analysis, scan_processing, report_generation, notification, data_cleanup
Dead Letter Queue	Jobs exceeding max_attempts (3) are moved to dead_letter status for manual review
Persistence	All jobs stored in NeDB with full lifecycle tracking (pending -> processing -> completed/dead_letter)
Offline Simulation	POST /async/simulate-offline produces jobs while consumer is stopped, demonstrating persistence

8.2 Event-Driven Architecture

The AsyncProcessor extends Node.js EventEmitter, emitting events (job:produced, job:completed, job:failed) that can be consumed by other system components. Each job execution is logged as a system event in NeDB, creating a complete audit trail of background processing.

9. Data Transformation & Interoperability

The DataTransformer service enables format conversion and API response restructuring for interoperability across systems.

Transformation	Endpoint / Method	Description
JSON to XML	GET /incidents/:id/xml	Converts incident to XML with xml2js Builder
XML to JSON	POST /transform/xml-to-json	Parses XML string into JSON object
JSON to JSON (External)	GET /incidents/:id/external	Restructures incident for external API format
JSON to STIX 2.1	GET /incidents/:id/stix	Converts to Structured Threat Information eXpression bundle
Response Wrapping	All endpoints	Standardized { success, data, meta } envelope
Flatten Nested Data	Internal service	Converts nested scan results to flat tabular format

10. Logging & Monitoring

Winston provides structured JSON logging with three transports: console (colorized), combined.log (all events), and error.log (errors only). Log files have 5MB rotation with 5-file retention.

10.1 Metrics Tracked

Metric	Description
Request Count	Total, success (2xx/3xx), error (4xx/5xx) counts
Response Time	Average and P95 latency in milliseconds
DB Queries	Count of SQLite and NeDB operations
Cache Performance	Hits, misses, sets with hit rate percentage
Uptime	Server uptime in seconds
Audit Trail	All CRUD actions with user, IP, timestamp in SQLite audit_log table

The GET /api/v3/metrics endpoint exposes all metrics in real-time. Combined with the audit log and system events, this provides full observability for debugging and maintenance.

11. Challenges and Reflections

11.1 Challenges Encountered

Dual Database Coordination: Managing data consistency across SQLite and NeDB required careful consideration of which data belongs in which store. Structured, relational data (users, incidents with foreign keys) fits naturally in SQLite, while flexible, variable-schema data (IOCs, events) benefits from NeDB's document model.

Cache Invalidation: Implementing tag-based cache invalidation was essential to avoid serving stale data after CRUD operations. The challenge was ensuring all relevant cache entries are cleared without over-invalidating, which would reduce cache effectiveness.

Async Job Reliability: Ensuring jobs persist across server restarts and handle failures gracefully required the dead-letter queue pattern. The retry mechanism with exponential backoff prevents infinite retry loops while giving transient failures a chance to recover.

Security Layering: Balancing security strictness with API usability required making authentication optional for read endpoints while mandatory for write operations. Rate limiting needed different thresholds for auth endpoints (stricter) vs. general API calls.

Native Module Compatibility: The initial choice of better-sqlite3 required native C++ compilation, which created deployment challenges. Switching to sql.js (pure JavaScript WebAssembly-based SQLite) resolved this while maintaining full SQL compatibility.

11.2 Reflections

Phase III demonstrates that a well-structured data management layer is the backbone of any enterprise application. The MVC architecture with clear separation of concerns (routes, controllers, services, models) makes the codebase maintainable and testable. Using both SQL and NoSQL databases in a single system highlights the strengths of each approach and shows that polyglot persistence is practical even in smaller projects. The caching layer proved that simple in-memory caching can provide dramatic performance improvements (1000x speedup) with minimal complexity, though production systems would benefit from Redis for distributed caching. Overall, Phase III successfully ties together data management, security, async processing, and observability into a cohesive system.